

Examen-SO-Modulo-II-2021.pdf



Darigod



Sistemas Operativos



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada**



[Accede al documento original](#)



Escuela de
Organización
Industrial

Contigo que evolucionas.
Contigo que lideras. Contigo que transformas.

**Esto es EOI.
Mismo propósito,
nueva energía.**



Descubre más aquí



EOI Escuela de
Organización
Industrial

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Examen SO Módulo II 2021

Ejercicio 1 [4 puntos]:

Desarrolle un programa en C utilizando llamadas al sistema y haciendo uso de los mecanismos que considere oportunos para copiar los archivos regulares con inodos impares. Se pasa como primer argumento el directorio origen (**deberá recorrer la jerarquía de subdirectorios indicando la profundidad deseada como segundo argumento**), como tercer argumento el directorio destino. Si no existe el directorio destino, debe de crearse. Deberá de crear un archivo donde se deberá:

1. Añadir una línea donde indica el número total de archivos copiados y la fecha actual.
2. Añadir una línea por cada uno de los archivos que se han copiado, debiendo incorporar también la ruta del archivo donde se encuentra.

Ejemplo:

fichero.txt

Ficheros totales copiados: 5 16-12-2021
ruta_del_archivo nombre_del_archivo

Ejercicio 2 [6 puntos]:

Crear un segundo programa en C que se ejecute de tal forma que lea números hasta que reciba un número menor que 0. Este programa deberá crear dos hijos los cuales deberán de ir realizando una serie de operaciones con cada uno de los números introducidos por el padre.

El padre enviará el número a los dos hijos que realizará el siguiente proceso:

- El primer hijo ejecutará una sumatoria de los números que vaya recibiendo por parte del padre.
- El segundo hijo ejecutará una factorización de los números que vaya recibiendo por parte del padre.

Finalmente el padre recibirá el total acumulado por parte de los dos hijos y creará un archivo donde guardará los siguientes resultados:

- Total sumatoria: 20
- Total factorización: 30
- Cambiar los permisos a 755.

Consideraciones:

Se debe tener en cuenta toda la gestión de errores posible.
No deben quedar procesos zombies.

Ejercicio 1:

```
#include <stdlib.h>
#include <unistd.h>
```

```

#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <dirent.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>

int cont = 0;

void recorrer_dir(const char *orig, int depth, const char *dest, const char *
path_actual) {

    DIR *d = opendir(orig);
    if (!d) {
        perror("opendir");
        return;
    }

    struct dirent *actual;

    while ((actual = readdir(d)) != NULL) {

        // saltar . y ..
        if (strcmp(actual->d_name, ".") != 0 && strcmp(actual->d_name, "..") !=
0){

            // construir ruta completa
            char ruta[1024];
            snprintf(ruta, sizeof(ruta), "%s/%s", orig, actual->d_name);

            struct stat atrib;
            if (stat(ruta, &atrib) == -1) {
                perror("stat");
            }
        }
    }
}

```

```

// Si es archivo regular
if (S_ISREG(atrib.st_mode)) {

    if (atrib.st_ino % 2 != 0) { // i-nodo impar
        // Copiar archivo
        int fd_src = open(ruta, O_RDONLY);
        if (fd_src < 0) {
            perror("open src");
        }

        // destino: dest + "/" + nombre_archivo
        char ruta_dest[1024];
        snprintf(ruta_dest, sizeof(ruta_dest), "%s/%s", dest, atrib.st_
d_name);

        int fd_dst = open(ruta_dest, O_WRONLY | O_CREAT | O_TRUNC
C, 0666);

        if (fd_dst < 0) {
            perror("open dst");
            close(fd_src);
        }

        char buffer[1024];
        int n;

        while ((n = read(fd_src, buffer, sizeof(buffer))) > 0) {
            if (write(fd_dst, buffer, n) != n) {
                perror("write");
                break;
            }
        }

        if (n < 0) {
            perror("read");
        }
    }
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
        close(fd_src);
        close(fd_dst);

        cont++;
    }
}
// Si es directorio
else if (S_ISDIR(atrib.st_mode)) {
    if (depth > 0) {
        recorrer_dir(ruta, depth - 1, dest, ruta);
    }
}

}

}

closedir(d);
}

int main(int argc, char *argv[]) {

    if (argc < 4) {
        fprintf(stderr, "Uso: %s <raiz> <profundidad> <dest>\n", argv[0]);
        exit(1);
    }

    char *orig = argv[1];
    int prof = atoi(argv[2]);
    char *dest = argv[3];

    // Comprobar/crear directorio destino
    struct stat st;
    if (stat(dest, &st) == -1) {
        if (errno == ENOENT) {
            if (mkdir(dest, 0777) == -1) {
                perror("mkdir");
                exit(1);
            }
        }
    }
}
```

```

    }
} else {
    perror("stat");
    exit(1);
}
}

// Abrir fichero de salida
int fd = open("fichero.txt", O_TRUNC | O_CREAT | O_RDWR, 0666);
if (fd < 0) {
    perror("open fichero.txt");
    exit(1);
}

// Recorrer directorio
recorrer_dir(orig, prof, dest, orig);

//Volvemos al principio del archivo para escribir cabecera
lseek(fd, 0, SEEK_SET);

// obtener fecha
time_t t = time(NULL);
struct tm *tm_info = localtime(&t);
char fecha[32];
strftime(fecha, sizeof(fecha), "%d-%m-%Y", tm_info);

char msg[256];
int len = snprintf(msg, sizeof(msg),
    "Ficheros totales copiados: %d  %s\n",
    cont, fecha);

if (write(fd, msg, len) != len) {
    perror("write cabecera");
}

close(fd);

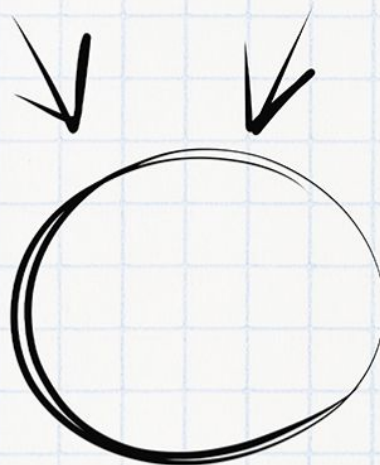
```


Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

```
    return 0;
}
```

Ejercicio 2

```
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <time.h>
#include <errno.h>

int num_hijos = 2;
int fin = 0; // 0 = seguir, 1 = terminar

int factorial(int n) {
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact *= i;
    return fact;
}

void child_handler(int sig) {
    (void)sig;
    while (waitpid(-1, NULL, WNOHANG) > 0) {}
}

void sigusr_handler(int sig) {
    (void)sig;
    fin = 1;
}
```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins?

→ Plan Turbo: barato

→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
int main(int argc, char *argv[]) {

    int pipe_envio_suma[2]; // padre → hijo suma
    int pipe_envio_fact[2]; // padre → hijo factorial
    int pipe_suma[2];       // hijo suma → padre
    int pipe_fact[2];       // hijo factorial → padre

    if (pipe(pipe_envio_suma) == -1 ||
        pipe(pipe_envio_fact) == -1 ||
        pipe(pipe_suma) == -1 ||
        pipe(pipe_fact) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < num_hijos; i++) {
        pid_t pid = fork();
        if (pid == -1) {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (pid == 0) { // HIJO

            struct sigaction sa;
            sa.sa_handler = sigusr_handler;
            sigemptyset(&sa.sa_mask);
            sa.sa_flags = 0;
            sigaction(SIGUSR1, &sa, NULL);

            if (i == 0) { // HIJO SUMA
                close(pipe_envio_fact[0]);
                close(pipe_envio_fact[1]);
                close(pipe_suma[0]);
                close(pipe_fact[0]);
                close(pipe_fact[1]);
                close(pipe_envio_suma[1]);
            }
        }
    }
}
```

```

int suma = 0;
while (!fin) {
    int dato;
    ssize_t n = read(pipe_envio_suma[0], &dato, sizeof(dato));
    if (n <= 0 || dato < 0) break;
    suma += dato;
}

write(pipe_suma[1], &suma, sizeof(suma));

close(pipe_envio_suma[0]);
close(pipe_suma[1]);
exit(EXIT_SUCCESS);
}
else { // HIJO FACTORIAL
    close(pipe_envio_suma[0]);
    close(pipe_envio_suma[1]);
    close(pipe_fact[0]);
    close(pipe_suma[0]);
    close(pipe_suma[1]);
    close(pipe_envio_fact[1]);

    int acu_fact = 0;
    while (!fin) {
        int dato;
        ssize_t n = read(pipe_envio_fact[0], &dato, sizeof(dato));
        if (n <= 0 || dato < 0) break;
        acu_fact += factorial(dato);
    }

    write(pipe_fact[1], &acu_fact, sizeof(acu_fact));

    close(pipe_envio_fact[0]);
    close(pipe_fact[1]);
    exit(EXIT_SUCCESS);
}
}

```

```

}

// PADRE

struct sigaction sa;
sa.sa_handler = child_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = SA_RESTART;
sigaction(SIGCHLD, &sa, NULL);

close(pipe_envio_suma[0]);
close(pipe_envio_fact[0]);
close(pipe_suma[1]);
close(pipe_fact[1]);

int x;
while (1) {
    printf("Insertar número (< 0 para fin): ");
    fflush(stdout);

    if (scanf("%d", &x) != 1) break;

    if (x < 0) {
        write(pipe_envio_suma[1], &x, sizeof(x));
        write(pipe_envio_fact[1], &x, sizeof(x));
        kill(0, SIGUSR1);
    }
    else{
        write(pipe_envio_suma[1], &x, sizeof(x));
        write(pipe_envio_fact[1], &x, sizeof(x));
    }

}

close(pipe_envio_suma[1]);
close(pipe_envio_fact[1]);

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
//Leemos e imprimimos
int suma_total = 0, fact_total = 0;
read(pipe_suma[0], &suma_total, sizeof(suma_total));
read(pipe_fact[0], &fact_total, sizeof(fact_total));

close(pipe_suma[0]);
close(pipe_fact[0]);

for (int i = 0; i < num_hijos; i++)
    wait(NULL);

int fd = open("resultados.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
char buf[256];
int len = snprintf(buf, sizeof(buf),
    "Total sumatoria: %d\nTotal factorizacion: %d\n",
    suma_total, fact_total);
write(fd, buf, len);
close(fd);

chmod("resultados.txt", 0755);

printf("Resultados guardados en resultados.txt\n");
printf("Total sumatoria: %d\n", suma_total);
printf("Total factorizacion: %d\n", fact_total);

return 0;
}
```